

說明：除 TextBox 外，像 DropDownList、RadioButton、CheckBox 等大多數控制項，皆可配合 ValidationMessage 及 ValidationSummary 方法產生警告訊息

HTML 輸出：

```
<input data-val="true" data-val-required="Name 必須輸入!" id="Name" name="Name"
  type="text" value="" />
<span class="field-validation-valid" data-valmsg-for="Name"
  data-valmsg-replace="true"></span>
```

ValidationMessage 的 HTML

```
<input data-val="true" data-val-required="Password 必須輸入!" id="Password"
  name="Password" type="text" value="" />
<span class="field-validation-valid" data-valmsg-for="Password"
  data-valmsg-replace="true"></span>
```

ValidationMessageFor 的 HTML

```
<div class="validation-summary-valid" data-valmsg-summary="true">
  <ul>
    <li style="display:none"></li>
  </ul>
</div>
```

ValidationSummary 的 HTML

範例 11-1 以 ValidationMessage 及 ValidationSummary 方法產生輸入驗證

以下用 ValidationMessage 及 ValidationSummary 方法驗證 TextBox 是否有輸入文字，否則提出警告。

step 01 在 User 模型將 Name 及 Password 屬性套用[Required(...)]，代表必須輸入，而 ErrorMessage 是驗證不通過時顯示的警告訊息

 Models/User.cs

```
using System.ComponentModel.DataAnnotations;

public class User
{
    public int Id { get; set; }
    [Required(ErrorMessage = "Name 必須輸入!")]
    public string Name { get; set; }
    [Display(Name = "暱稱")]
    public string Nickname { get; set; }
    [Required(ErrorMessage = "Password 必須輸入!")]
    [DataType(DataType.Password)]
    public string Password { get; set; }
    ...
}
```

屬性套用[Required]後，會對 TextBox 產生什麼影響？以下說明。

+ 未套用[Required]，TextBox 的 HTML 輸出不包含驗證

```
<input id="Name" name="Name" type="text" value="" />
<input id="Password" name="Password" type="text" value="" />
```

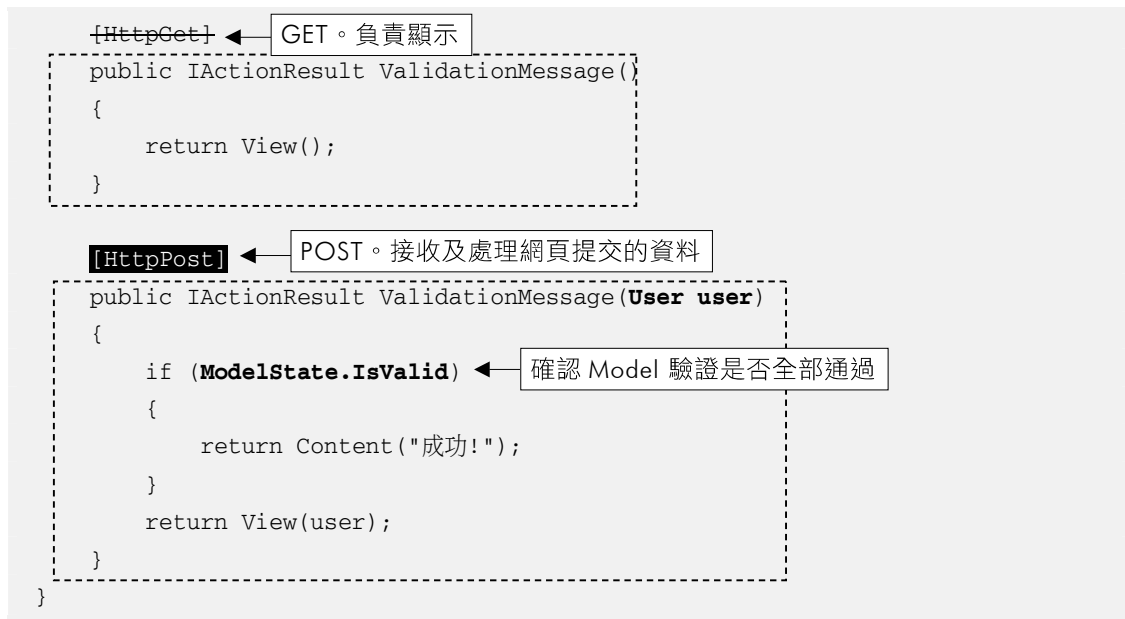
+ 套用[Required]後，HTML 多了 data-val 及 data-val-required 屬性，但仍必須配合 ValidationMessage 才會產生驗證的警告文字

```
<input data-val="true" data-val-required="Name 必須輸入!" id="Name" name="Name"
    type="text" value="" />
<input data-val="true" data-val-required="Password 必須輸入!" id="Password"
    name="Password" type="text" value="" />
```

step 02 在 HtmlHelpers 控制器新增兩個 ValidationMessage 方法，第一個是負責網頁顯示（GET），第二個是負責接收及處理網頁提交的資料（POST）

 Controllers/HtmlHelpersController.cs

```
public class HtmlHelperController : Controller
{
```



說明：Action 預設就是[HttpGet]，不需特別標示。標示只是為了對比[HttpPost]，指出它們是負責不同的請求方法

step 03 在檢視中加入 ValidationMessage 及 ValidationSummary 方法，驗證 TextBox 是否有輸入，否則在 Submit 時就會顯示警告訊息

 Views/HtmlHelpers/ValidationMessage.cshtml

```

@model User
...
@using (Html.BeginForm())
{
    @Html.Label("Name")
    @Html.TextBox("Name")
    @Html.ValidationMessage("Name")
    <br />

    @Html.LabelFor(m => m.Password)
    @Html.PasswordFor(m => m.Password)
    @Html.ValidationMessageFor(m => m.Password)
    <br />
    <input type="submit" value="Submit" class="btn btn-primary" />
    @Html.ValidationSummary()
}

```

說明：

1. `ValidationMessage` 方法通常放在要驗證的控制項後，而 `Validation Summary` 方法放在 `<form>` 中的任何位置都行，但通常放在前頭或末段
2. 若驗證警告文字想要套用 Bootstrap 文字顏色，語法如下：

```
@Html.ValidationMessage("Name", "", new { @class = "text-danger" })
@Html.ValidationMessageFor(m => m.Password, "", new { @class = "text-danger" })
@Html.ValidationSummary(false, "", new { @class = "text-danger" })
```

11-3-14 `Html.Editor()` & `Html.EditorFor()` 方法

`Editor` 方法是用來產生 `<input type="xxx" ...>` 輸入控制項，但不同資料型別，會使得產出的控制項也有差異。`Editor` 方法會產生哪種控制項，實際是受到 `Model` 屬性的兩點影響：一是資料型別，二是套用 `[DataType(DataType.xxx)]` 的類型。

表 11-2 `Editor` 方法對不同資料型別的 HTML 輸出

資料型別 / <code>DataTypeAttribute</code>	<code>Editor</code> 方法輸出的 HTML element
string	<code><input type="text" ></code>
int	<code><input type="number" ></code>
decimal, float	<code><input type="number" ></code>
boolean	<code><input type="checkbox" ></code>
Enum 列舉	<code><input type="text" ></code>
<code>[DataType(DataType.Password)]</code>	<code><input type="password" ></code>
<code>[DataType(DataType.Email)]</code>	<code><input type="email" ></code>
<code>[DataType(DataType.Date)]</code>	<code><input type="date" ></code>
<code>[DataType(DataType.DateTime)]</code>	<code><input type="datetime" ></code>
<code>[DataType(DataType.Currency)]</code>	<code><input type="text" ></code>
<code>[DataType(DataType.MultilineText)]</code>	<code><textarea> ... </textarea></code>